EduCert Verify

AI-Assisted Certificate Authenticity & QR Consistency Verification System

Abstract

EduCert Verify is a Flask-based academic project that analyzes an uploaded certificate image, decodes its QR code, extracts visible certificate text through OCR, and compares credential data against an issuer-side registry. The system generates a verification result, score, reasons and an audit log. The design separates consistency checking from true issuer authentication: a production system should connect to an issuer-controlled verification service or trusted registry.

1. Problem Statement

Digital certificates are easy to copy or modify as images and PDFs. Manual verification can be slow and inconsistent. A verifier needs a practical mechanism to compare machine-readable credential data with the certificate's visible information and a trusted registration record.

2. Objectives

1) Scan QR codes automatically. 2) Extract certificate information using OCR. 3) Match QR fields with a registry record. 4) Detect mismatch and revoked records. 5) Provide an admin dashboard and REST API. 6) Maintain verification logs.

3. Architecture

User → Flask Web UI → Upload Handler → OpenCV QR Decoder + Tesseract OCR → Verification Engine → SQLite Registry → Result & Audit Log. The REST API exposes the same verification engine to external applications.

4. Technology Stack

Layer	Technology
Frontend	HTML, CSS, JavaScript
Backend	Python Flask
QR	OpenCV QRCodeDetector
OCR	Tesseract + pytesseract
Database	SQLite + Flask-SQLAlchemy
API	REST/JSON
Testing	Postman
Report	ReportLab

5. Functional Modules

Certificate Upload: accepts common image formats and limits upload size. QR Scanner: decodes JSON payload. OCR: extracts visible text when Tesseract is available. Registry: stores certificate ID, name, course, issuer, date, URL, hash and status. Verification Engine: compares QR payload with the registry and produces a result. Admin: registers and revokes certificates. API: supports health, lookup and verification. Logs: stores verification outcomes for auditing.

6. Verification Algorithm

```
1. Receive certificate image.
2. Decode QR code.
3. Parse QR JSON.
4. Read certificate_id.
5. Query issuer registry.
6. If no record: UNREGISTERED.
7. Compare QR fields with registry.
8. Calculate matched-field percentage.
9. Check registry status.
10. Run OCR for visible text.
11. Save verification log.
12. Return result and reasons.
```

7. Result States

State	Meaning
VERIFIED	All available QR fields match an active registry record.
PARTIAL_MATCH	Some fields match, but a mismatch or missing field exists.
MISMATCH	QR data does not sufficiently match the registered record.
REVOKED	The registry record exists but is not active.
UNREGISTERED	No matching certificate ID exists in the registry.
QR_NOT_FOUND	No readable QR code was detected.

8. Database Design

Certificate(certificate_id, student_name, course_name, issuer, issue_date, credential_url, certificate_hash, status, created_at). VerificationLog(certificate_id, result, score, details_json, created_at).

9. API Endpoints

GET /api/health — service health. POST /api/verify — upload certificate image in multipart field 'certificate'. GET /api/certificates/{certificate_id} — retrieve registry status and details.

10. Testing

The included sample certificate contains a QR payload matching the seeded EduSkills-style registry record. Uploading it should produce VERIFIED when QR decoding succeeds. A certificate with a changed QR certificate ID should produce UNREGISTERED; a registered ID with altered QR fields should produce MISMATCH or PARTIAL_MATCH. Revoking a record should produce REVOKED.

11. Security and Ethical Considerations

The local demo uses simple admin credentials and SQLite for educational purposes. Production use requires strong password hashing, HTTPS, CSRF protection, rate limiting, secure file storage, malware scanning, strict authorization, audit controls and issuer-controlled verification. A QR code alone is not proof of authenticity because an attacker can create a QR code containing false information.

12. Future Enhancements

Issuer API integration; cryptographic digital signatures; W3C Verifiable Credentials; blockchain anchoring; PDF text extraction; layout-aware OCR; duplicate detection; certificate image tamper analysis; multilingual OCR; mobile camera scanning; cloud deployment.

13. Conclusion

EduCert Verify demonstrates an end-to-end certificate verification workflow combining QR scanning, OCR, registry comparison, administrative control and API access. It provides a strong MCA project foundation while keeping the boundary between consistency checking and genuine issuer authentication explicit.